

MINIMALES HTML5-DOKUMENT

```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="UTF-8">
    <title>Titel</title>
  </head>
  <body>Hello World</body>
</html>
```

Aufbau eines Elements

```
<tag attr="wert">Inhalt</tag>
```

Start-Tag + Attribut=Wert + Inhalt + End-Tag.

Leere (void) Elemente

Kein Inhalt, kein End-Tag:

```
<br> <img> <meta> <hr> <input>
```

BLOCK VS. INLINE

Block-Elemente stapeln untereinander, Inline-Elemente fließen im Text:

	Block	Inline
Breite	volle Zeile	nur Inhalt
Umbruch	neue Zeile	im Textfluss
width/height	wirken	wirken nicht
Beispiele	div p h1 ul li table	span a b em strong img

Head-Tags

```
<title> Tab/Validität · <meta charset> Umlaute ·
<link rel="stylesheet"> CSS · <script> JS
```

Hyperlink & URL

```
<a href="ziel">Text</a>
protokoll://host:port/pfad?query#fragment
```

LISTEN (OHNE CSS)

ul / ol

```
<ul> ungeordnet (Punkte) · <ol> geordnet (1.2.3.). Einträge je
<li> .
```

dl — Definitionsliste (3. Art)

```
<dl><dt>Begriff</dt><dd>Erklärung</dd></dl>
```

Falle: Fragment vervollständigen — innerste Tags zuerst schließen (`</span>` vor `</p>`).

Regel-Syntax

```
selektor { eigenschaft: wert; }  
/* p { color: red; } */
```

Einbinden

extern `<link rel=stylesheet>` · intern `<style>` · inline `style=""`

Kaskade: User-Agent < User < Autor < **!important**

SELEKTOREN (BODY: OL#OL1.AWE · P4.AWE.MARIA)

Selektoren bestimmen, welche Elemente eine Regel trifft:

Selektor	Matcht
#p1 · .awe	ID p1 · Klasse awe → ol1, p4
ol[class="awe"]	nur exakt "awe" → ol1
[class~="awe"]	Wort in Liste → ol1, p4
[a^= / \$= / *]=	beginnt / endet / enthält
E F · E>F	Nachfahre · direktes Kind
E+F · E~F	direkter Nachbar · alle folgenden Geschwister

Box-Model

innen → außen: **content → padding → border → margin**

Gesamt = width + 2·padding + 2·border + 2·margin

position

static (default): top/left wirkungslos

relative: verschoben von Normalposition

absolute: rel. zum pos. Vorfahren, aus dem Fluss

Fallen: `border:5px` wirkungslos (style fehlt!) · `ol first-child` ohne Doppelpunkt = sucht <first-child>-Tag → 0 Treffer · `[a="x"]` exakt vs `[a~="x"]` Wortliste.

NEW VS. PLAIN CALL — DER TRACE-KERN

Ob mit oder ohne `new` aufgerufen wird, ändert `this` und die Rückgabe:

Aufruf	this	Rückgabe
<code>new Foo()</code>	neues Objekt	Objekt; primitiver return ignoriert
<code>Foo()</code>	global/undefined	der return-Wert zählt

Scope: `let a` in der Funktion verdeckt globales `a` (Shadowing). Fehlt `let` → globale Variable. Globales `b` wird von jedem Aufruf überschrieben → letzter Aufruf gewinnt.

DOM ERZEUGEN & EINFÜGEN

```
const ul = document.createElement("ul");
const li = document.createElement("li");
li.setAttribute("land", "de");
ul.appendChild(li);
document.getElementById("inhalt").appendChild(ul);
```

Gleichheit

- `==` wandelt Typen um → `42=="42"` true
- `===` prüft Typ mit → `42===42` false

AJAX · JSON · Sandbox

`await fetch(url)` → `.text().json()`

Sandbox verhindert: Dateisystem, fremde Origin (Same-Origin), OS.

JS EINBINDEN & BROWSER-APIS

JS in HTML einbinden

```
<script src="app.js"></script> // extern
<script> ...code... </script> // intern
```

Standard-Browser-API-Objekte

`document` (DOM), `console` (Log), `window` (Fenster), `fetch` (HTTP), `localStorage`.

Merke: `new` + primitiver `return` = Objekt gewinnt · Plain-Call = Wert gewinnt.

**Zwei
Aufgaben
eines
Webservers**

- 1)
HTTP-
Requests
entgegennehmen
&
beantworten.
- 2)
Ressourcen
ausliefern
—
statisch
direkt
oder
dynamisch
an
Programmcode
delegieren.

PROZESSMODELLE

Wie Server-Aufgabe und Programmcode zusammenlaufen:

Modell	Beschreibung
Getrennte Prozesse = CGI	neuer Prozess pro Request — isoliert, aber langsam
Selber Prozess (In-Process)	Code im Server-Prozess (Modul/NodeJS) — schnell, weniger isoliert

HTTP-Status

2xx Erfolg (200) · **3xx** Umleitung (301/302/304)

4xx Client (404/403/400) · **5xx** Server (500/503)

URL-Arten

absolut `https://x.de/a` · root-rel. `/img/a`

dokument-rel. `../a` · protokoll-rel. `//cdn/a`

HTTP-Methoden

GET Ressource anfordern (Parameter in der URL) · **POST** Daten senden (im Body, z.B. Formular) · **PUT / DELETE** anlegen-ersetzen / löschen
· **HEAD** nur Header.

Falle: `/start` ist **relativ** (root-relativ), nicht absolut. Vorteil relativer URLs: portabel, kürzer.

Server-Grundgerüst

```
const app = require("express")();
app.get("/", (req, res) => {
  res.send("Hi");
});
app.listen(3000);
```

Forward vs. Redirect

Forward: serverintern, URL bleibt, 1 Roundtrip.

Redirect: 302 + neue URL, Browser holt neu, URL ändert sich (2 Roundtrips).

MIDDLEWARE — PARAM == 42 → /SPEZIAL

```
app.use("/anfrage", (req, res, next) => {
  if (req.body.param == "42") res.redirect("/spezial");
  else next(); // sonst hängt die Anfrage!
});
```

Cookies

Server: `Set-Cookie` → Browser speichert → schickt automatisch bei jedem Request mit. Macht zustandsloses HTTP zustandsbehaftet.

WebSocket · Scopes

WebSocket: bidirektional & persistent (vs. Request-Response).

NodeJS-Scopes: Modul-lokal + global.

TEMPLATES · MODEL-1/2 · NPM**Templates / Pragmas**

Template = HTML-Gerüst mit Platzhaltern (**Pragmas**). Beim **Rendern** werden die Pragmas durch Daten ersetzt → Ergebnis ist fertiges, statisches HTML für den Client.

Model-1: View + Logik gemischt. **Model-2** (MVC): Controller → View getrennt → bessere Kapselung.

npm = Paketmanager: `npm install express`, package.json.

JavaScript-Sandbox

JS isoliert im Seitenkontext: nur DOM + Web-APIs. Kein Dateisystem, keine fremde Origin (Same-Origin), kein OS.

Sichere Cookies & HTTPS

HttpOnly (kein JS) · **Secure** (nur HTTPS) · **SameSite** (CSRF).

HTTPS/TLS: verschlüsselt, integer, authentifiziert.

TYPISCHE ANGRIFFE

Die drei Klassiker der Web-Sicherheit:

Angriff	Kurz
XSS	fremdes JS in die Seite eingeschleust
CSRF	Aktion im Namen des eingeloggten Nutzers
SQL-Injection	Schad-SQL über Eingabefelder → Prepared Statements

SEO — DAY6 (LAUT DOZENT KLAUSURRELEVANT)

On-Page

semantisches HTML (h1→h6), `<title>`, meta description, alt-Texte, sprechende URLs, interne Links.

Technisch & Off-Page

robots.txt (crawlen), sitemap.xml, `rel="canonical"`, JSON-LD, hreflang.

Off-Page: Backlinks als Reputationssignal.